

JUnit Framework

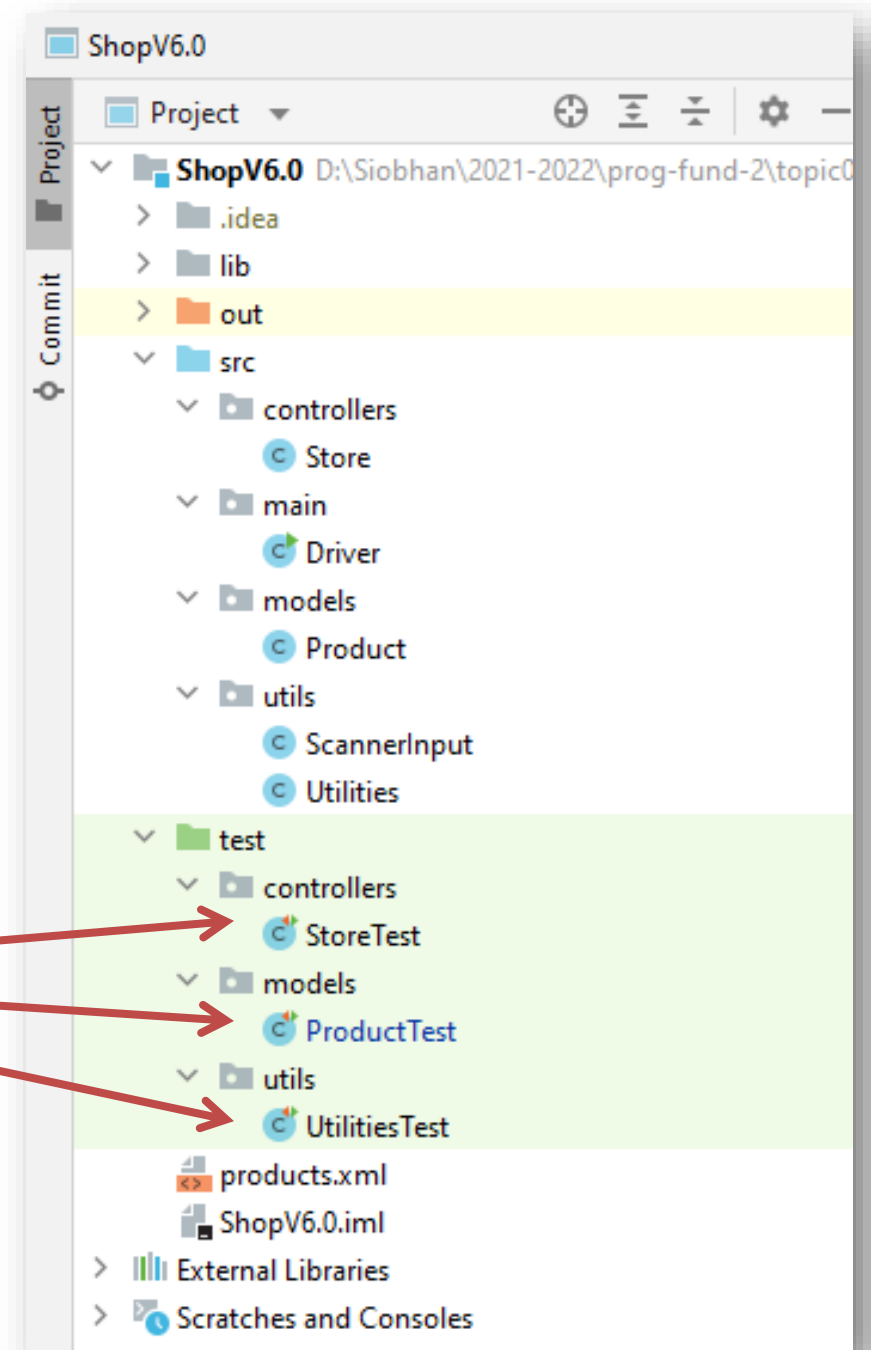
Terminology: assertions, annotations, fixtures

Produced Dr. Siobhán Drohan
by: Mairead Meagher
 Siobhan Roche

Topic List

- General Terminology
- Assertions
- Annotations
- Fixtures

JUnit Test Class /
Test Case



Setting up
the **test
fixture**

Tearing
down the
test fixture

Nesting
similar
tests
together

Test
Methods

```
ProductTest.java x
1  package models;
2
3  import ...
10
11  class ProductTest {
12
13      private Product productBelow, productExact, productAbove, productZero;
14
15      @BeforeEach
16      void setUp() {
17          //name, 19 chars, code 999, unitCost 1, inCurrentProductLine true.
18          productBelow = new Product( productName: "Television 42Inches", productCode: 999, unitCost: 1, inCurrentProductLine: true);
19          //name, 20 chars, code 1000, unitCost 999, inCurrentProductLine true.
20          productExact = new Product( productName: "Television 50 Inches", productCode: 1000, unitCost: 999, inCurrentProductLine: true);
21          //name, 21 chars, code 10000, unitCost 1000, inCurrentProductLine false.
22          productAbove = new Product( productName: "Television 60 Inches.", productCode: 10000, unitCost: 1000, inCurrentProductLine: false);
23          //name, 0 chars, code 9999, unitCost 0, inCurrentProductLine false.
24          productZero = new Product( productName: "", productCode: 9999, unitCost: 0, inCurrentProductLine: false);
25      }
26
27      @AfterEach
28      void tearDown() { productBelow = productExact = productAbove = productZero = null; }
29
30
31      @Nested
32      class Getters {
33
34
35          @Test
36          void getProductName() {...}
37
38
39          @Test
40          void getUnitCost() {...}
41
42
43          @Test
44          void getProductCode() {...}
45
46
47
48
49
50
51
52
53
54
55
56
57
58
```

Topic List

- General Terminology

- Assertions

- Annotations

- Fixtures

First you import the **Assertions** class from **org.junit...**

Test Methods then contain assertions

```
ProductTest.java x
9  import static org.junit.jupiter.api.Assertions.*;
10
11  class ProductTest {
12
13      private Product productBelow, productExact, productAbove, productZero;
14
15      @BeforeEach
16      void setUp() {...}
17
18      @AfterEach
19      void tearDown() { productBelow = productExact = productAbove = productZero = null; }
20
21      @Nested
22      class Getters {
23
24          @Test
25          void getProductName() {
26              assertEquals( expected: "Television 42Inches", productBelow.getProductName());
27              assertEquals( expected: "Television 50 Inches", productExact.getProductName());
28              assertEquals( expected: "Television 60 Inches", productAbove.getProductName());
29              assertEquals( expected: "", productZero.getProductName());
30          }
31
32          @Test
33          void getUnitCost() {...}
34
35          @Test
36          void getProductCode() {...}
37
38          @Test
39          void isInCurrentProductLine() {
40              assertTrue(productBelow.isInCurrentProductLine());
41              assertTrue(productExact.isInCurrentProductLine());
42              assertFalse(productAbove.isInCurrentProductLine());
43              assertFalse(productZero.isInCurrentProductLine());
44          }
45      }
46  }
```

The screenshot shows a web browser window displaying the JUnit 5 API documentation for the `Assertions` class. The browser's address bar shows the URL `https://junit.org/junit5/docs/current/api/org.junit.jupiter.api/org.junit.jupiter.api/Assertions.html`. The page has a green header with navigation links: OVERVIEW, MODULE, PACKAGE, CLASS (selected), USE, TREE, DEPRECATED, INDEX, and HELP. Below the header, there's a search bar and a summary section with links for NESTED, FIELD, CONSTR, and METHOD. The main content area shows the module `org.junit.jupiter.api` and package `org.junit.jupiter.api`. The class `Assertions` is listed as extending `Object`. The documentation includes a description of the class as a collection of utility methods for asserting conditions in tests, a note about the `AssertionFailedError` exception, and sections for Object Equality, Kotlin Support, and Preemptive Timeouts.

Module `org.junit.jupiter.api`
Package `org.junit.jupiter.api`

Class `Assertions`

`java.lang.Object`
`org.junit.jupiter.api.Assertions`

```
@API(status=STABLE, since="5.0")  
public class Assertions  
    extends Object
```

`Assertions` is a collection of utility methods that support asserting conditions in tests.

Unless otherwise noted, a *failed* assertion will throw an `AssertionFailedError` or a subclass thereof.

Object Equality

Assertion methods comparing two objects for *equality*, such as the `assertEquals(expected, actual)` and `assertNotEquals(unexpected, actual)` variants, are *only* intended to test equality for an (un-)expected value and an actual value. They are not designed for testing whether a class correctly implements `Object.equals(Object)`. For example, `assertEquals()` might immediately return true when provided the same object for the expected and actual values, without calling `equals(Object)` at all. Tests that aim to verify the `equals(Object)` implementation should instead be written to explicitly verify the `Object.equals(Object)` contract by using `assertTrue()` or `assertFalse()` — for example, `assertTrue(expected.equals(actual))`, `assertTrue(actual.equals(expected))`, `assertFalse(expected.equals(null))`, etc.

Kotlin Support

Additional `Kotlin` assertions can be found as *top-level functions* in the `org.junit.jupiter.api` package.

Preemptive Timeouts

The various `assertTimeoutPreemptively()` methods in this class execute the provided executable or supplier in a different thread than that of the calling code. This behavior can lead to undesirable side effects if the code that is executed within the executable or supplier relies on `ThreadLocal` storage.

One common example of this is the transactional testing support in the Spring Framework. Specifically, Spring's testing support binds transaction state to the current thread (via a

The Assertions Class

- To check if code is behaving as you expect, you use an assertion.
- An assertion is a simple method call that verifies that something is true.
- The Assertions class Contains a set of assertion methods useful for writing JUnit tests.
- Only failed assertions are recorded i.e. an **AssertionFailedError** is thrown and handled by JUnit.

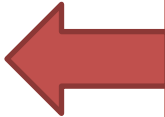
The Assertions Class

These methods can be used directly:

```
Assertions.assertEquals(...);
```

However, they read better if they are referenced through a static import:

```
import static org.junit.jupiter.api.Assertions.*;  
//some code  
assertEquals(...);
```



We will
use this
approach.

Some common Assert methods (1)

Method Summary	
static void	<u>assertEquals</u> (double expected, double actual, double delta) Asserts that two doubles are equal to within a positive delta.
static void	<u>assertEquals</u> (long expected, long actual) Asserts that two longs are equal.
static void	<u>assertEquals</u> (<u>Object</u> expected, <u>Object</u> actual) Asserts that two objects are equal.
static void	<u>assertNotEquals</u> (double unexpected, double actual, double delta) Asserts that two doubles are not equal to within a positive delta.
static void	<u>assertNotEquals</u> (<u>Object</u> unexpected, <u>Object</u> actual) Asserts that two objects are not equals.
static void	<u>assertNotSame</u> (<u>Object</u> unexpected, <u>Object</u> actual) Asserts that two objects do not refer to the same object.
static void	<u>assertSame</u> (<u>Object</u> expected, <u>Object</u> actual) Asserts that two objects refer to the same object.

Some common Assert methods (2)

Method Summary	
static void	<u>assertNotNull</u> (<u>Object</u> object) Asserts that an object isn't null.
static void	<u>assertNotNull</u> (<u>String</u> message, <u>Object</u> object) Asserts that an object isn't null.
static void	<u>assertNull</u> (<u>Object</u> object) Asserts that an object is null.
static void	<u>assertFalse</u> (boolean condition) Asserts that a condition is false.
static void	<u>assertTrue</u> (<u>String</u> message, boolean condition) Asserts that a condition is true.
static void	<u>fail</u> () Fails a test with no message.
static void	<u>fail</u> (<u>String</u> message) Fails a test with the given message.

assertEquals

assertTrue

assertFalse

```
ProductTest.java x
9  import static org.junit.jupiter.api.Assertions.*;
10
11  class ProductTest {
12
13      private Product productBelow, productExact, productAbove, productZero;
14
15      @BeforeEach
16      void setUp() {...}
17
18
19
20
21
22
23
24
25
26
27      @AfterEach
28      void tearDown() { productBelow = productExact = productAbove = productZero = null; }
29
30
31
32      @Nested
33      class Getters {
34
35          @Test
36          void getProductName() {
37              assertEquals( expected: "Television 42Inches", productBelow.getProductName());
38              assertEquals( expected: "Television 50 Inches", productExact.getProductName());
39              assertEquals( expected: "Television 60 Inches", productAbove.getProductName());
40              assertEquals( expected: "", productZero.getProductName());
41          }
42
43          @Test
44          void getUnitCost() {...}
45
46
47
48
49          @Test
50          void getProductCode() {...}
51
52
53
54
55          @Test
56          void isInCurrentProductLine() {
57              assertTrue(productBelow.isInCurrentProductLine());
58              assertTrue(productExact.isInCurrentProductLine());
59              assertFalse(productAbove.isInCurrentProductLine());
60              assertFalse(productZero.isInCurrentProductLine());
61          }
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
}
```

Topic List

- General Terminology
- Assertions
- Annotations
- Fixtures

These
methods
have
Annotations

@someword

```
ProductTest.java x
9  import static org.junit.jupiter.api.Assertions.*;
10
11  class ProductTest {
12
13      private Product productBelow, productExact, productAbove, productZero;
14
15      @BeforeEach
16      void setUp() {...}
17
18      @AfterEach
19      void tearDown() { productBelow = productExact = productAbove = productZero = null; }
20
21      @Nested
22      class Getters {
23
24          @Test
25          void getProductName() {
26              assertEquals( expected: "Television 42Inches", productBelow.getProductName());
27              assertEquals( expected: "Television 50 Inches", productExact.getProductName());
28              assertEquals( expected: "Television 60 Inches", productAbove.getProductName());
29              assertEquals( expected: "", productZero.getProductName());
30          }
31
32          @Test
33          void getUnitCost() {...}
34
35          @Test
36          void getProductCode() {...}
37
38          @Test
39          void isInCurrentProductLine() {
40              assertTrue(productBelow.isInCurrentProductLine());
41              assertTrue(productExact.isInCurrentProductLine());
42              assertFalse(productAbove.isInCurrentProductLine());
43              assertFalse(productZero.isInCurrentProductLine());
44          }
45      }
46  }
```

What are Annotations?

- Annotations provide data about a program that is not part of the program itself. They have no direct effect on the operation of the code they annotate.
- Annotations can be applied to a program's declarations of classes, fields, methods, and other program elements.

What are Annotations?

- Annotations have a number of uses, among them:
 - Information for the compiler — Annotations can be used by the compiler to detect errors or suppress warnings.
 - Compiler-time and deployment-time processing — Software tools can process annotation information to generate code, XML files, and so forth.
 - Runtime processing — Some annotations are available to be examined at runtime.

JUnit 5 Annotations

Import the required Annotation class(es) from org.junit.

@BeforeEach runs the method before each test.

@AfterEach runs the method after each test.

@Nested
Groups together a series of methods into a nested class.

@Test
identifies that a method is a test method.

```
ProductTest.java
3  import org.junit.jupiter.api.AfterEach;
4  import org.junit.jupiter.api.BeforeEach;
5  import org.junit.jupiter.api.Nested;
6  import org.junit.jupiter.api.Test;
7
8  import static org.junit.jupiter.api.Assertions.*;
9
10 class ProductTest {
11
12     private Product productBelow, productExact, productAbove, productZero;
13
14     @BeforeEach
15     void setUp() {...}
16
17
18
19
20
21
22
23
24
25     @AfterEach
26     void tearDown() { productBelow = productExact = productAbove = productZero = null; }
27
28
29
30
31     @Nested
32     class Getters {
33
34
35         @Test
36         void getProductName() {...}
37
38
39
40
41
42         @Test
43         void getUnitCost() {...}
44
45
46
47
48
49
50         @Test
51         void getProductCode() {...}
52
53
54
55
56
57         @Test
58         void isInCurrentProductLine() {...}
59
60     }
61 }
```

Other Useful **JUnit5** Annotations

@BeforeAll public void method()	Will execute the method once, before the start of all tests in the current class.
@AfterAll public void method()	Will execute the method once, after all tests in the current class have finished.
@Disabled public void method()	Used to disable a test class or test method.

Topic List

- General Terminology
- Assertions
- Annotations
- Fixtures

```
ProductTest.java x
3  import org.junit.jupiter.api.AfterEach;
4  import org.junit.jupiter.api.BeforeEach;
5  import org.junit.jupiter.api.Nested;
6  import org.junit.jupiter.api.Test;
7
8  import static org.junit.jupiter.api.Assertions.*;
9
10 class ProductTest {
11
12     private Product productBelow, productExact, productAbove, productZero;
13
14     @BeforeEach
15     void setUp() {
16         //name, 19 chars, code 999, unitCost 1, inCurrentProductLine true.
17         productBelow = new Product( productName: "Television 42Inches", productCode: 999, unitCost: 1, inCurrentProductLine: true);
18         //name, 20 chars, code 1000, unitCost 999, inCurrentProductLine true.
19         productExact = new Product( productName: "Television 50 Inches", productCode: 1000, unitCost: 999, inCurrentProductLine: true);
20         //name, 21 chars, code 10000, unitCost 1000, inCurrentProductLine false.
21         productAbove = new Product( productName: "Television 60 Inches.", productCode: 10000, unitCost: 1000, inCurrentProductLine: false);
22         //name, 0 chars, code 9999, unitCost 0, inCurrentProductLine false.
23         productZero = new Product( productName: "", productCode: 9999, unitCost: 0, inCurrentProductLine: false);
24     }
25
26     @AfterEach
27     void tearDown() {
28         productBelow = productExact = productAbove = productZero = null;
29     }
30 }
```

Test fixture
fields.

Setting up the
test fixture.

Tearing down
the test
fixture.

Fixtures

- A **Fixture** is a fixed state of a set of objects used as a baseline for running tests.
- Test fixtures allow tests to share common test data.
- The purpose of a test fixture is to ensure that there is a fixed environment in which tests are run so that results are repeatable.
- It includes:
 - setUp() method which runs before every test method.
 - tearDown() method which runs after every test method.

**Any
Questions?**

